

第 09 章 ACL 与协作目录

从 ACL 条目到有效权限：把 access、default、mask、继承与多身份验收放进同一条证据链。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版

⋮ {navigation-page}

本章阅读导航

先抓住一条主线：ACL 不是“给用户写一行 `rwX` 就结束”，而是从主体匹配开始，经 `mask` 计算得到 `effective permissions`，再落到当前对象、未来新对象和真实身份功能测试。

::: {.model-grid}

01识别对象与主体确认文件、目录、owner、owning group 与 named entry

02区分 `access / default`当前访问与未来创建不是同一状态

03计算 `effectivenamed user` 与 `group class` 还要经过 `mask`

04执行最小修改新增、修改、删除时控制条目和递归范围

05验证继承时点已有对象、新文件和新目录分别取证

06多身份验收同时证明允许动作、拒绝动作和恢复依据

:::

::: {.navigation-columns}

::: {.nav-col}

专题地图

类型	专题
知识专题	ACL 对象、条目与作用时点
知识专题	访问匹配、 <code>mask</code> 与 <code>effective</code>
操作专题	<code>ls</code> 、 <code>stat</code> 与 <code>getfacl</code> 取证
操作专题	<code>setfacl</code> 新增、修改、删除与 <code>mask</code> 控制
操作专题	<code>default ACL</code> 与创建时继承
操作专题	协作目录的当前、历史与未来状态
操作专题	ACL 复制、备份与恢复
诊断专题	条目存在但仍然 <code>Permission denied</code>
经典任务	建设协作目录与修复 <code>mask</code>

:::

::: {.nav-col}

阅读时持续回答

1. 当前判断的是 access ACL，还是 default ACL？
2. 进程会匹配 owner、named user、group class 还是 other？
3. 该条目是否受 mask 限制？
4. `#effective:` 与名义权限是否一致？
5. 目标是已有对象还是未来新对象？
6. `chmod` 是否改变了 mask？
7. 查询证据能证明什么，不能证明什么？
8. 哪个真实身份动作才是最终验收？

...

...

....

第 09 章 · 正文

ACL 与协作目录：声明的权限为何不一定真正生效 传统 owner、group、other 三类权限适合表达多数单一团队场景，但真实目录常常同时存在“项目组可协作、审计员只读、个别人员需要额外权限”等要求。ACL 在不改变主要 owner 与 owning group 的前提下增加 named user、named group 和 mask，使授权能够精确到额外主体。本章最重要的不是记住 ``setfacl`` 选项，而是避免三种误判：****ACL 条目存在不等于有效权限存在；default ACL 存在不等于已有对象已经改变；命令成功不等于目标身份能够完成业务动作。**** 因此，本章沿着“对象与条目 → access/default → mask/effective → 查询与修改 → 继承 → 多身份验证 → 诊断与恢复”推进。传统模式位、umask、setgid 和 sticky 位已在第 08 章《传统权限、umask 与特殊权限位》中建立；本章只解释它们与 ACL 的接口。sudo 属于第 10 章《sudo 与最小特权授权》；SELinux 属于后续安全章节，本章在证据闭环后只指出分流方向，不提前展开。

当前对象存储了哪些 ACL 条目？

目标身份会匹配哪一种主体类别？

mask 后真正剩下哪些 effective permissions？

失败对象是历史文件、当前目录，还是未来新对象？

结构证据之后，哪个真实动作才能完成最终验收？

概念Access ACL 是文件或目录当前参与自主访问判断的规则集合。它描述 owner、named user、owning group、named group、mask 与 other 在“现在访问这个对象”时如何工作。目录上出现 default ACL，并不会替代当前目录的 access ACL；判断当前能否进入或读写，仍要读取 access 部分。

概念Default ACL 只能附着在目录上，是创建新对象时使用的 ACL 模板。它不会回溯修改已经存在的文件和子目录，也不直接决定父目录本身能否被访问。要验证 default ACL，必须真正创建新文件和新目录，再读取它们的 access ACL。

概念ACL entry 由主体类型、可选 qualifier 与 ``rwx`` 权限组成。``user::``、``group::``、``other::`` 是基础条目；

``user:alice:`` 与 ``group:qa:`` 是额外主体条目；``mask::`` 则描述整个 group class 的有效权限上限。读输出时要先

判断“这一行代表谁”，再谈权限。

概念ACL mask 不是一个新的用户或组，而是 named user、owning group 与所有 named group 的统一上限。提高 mask 可能同时放宽多个条目的 effective 权限，收窄 mask 也可能让多个主体一起失去权限，因此修改 mask 前必须审阅整个 group class。

概念Effective permissions 是条目权限经过匹配规则与 mask 约束后真正参与访问判断的结果。`user:analyst:rw-` 与 `mask::r--` 同时存在时，analyst 实际只有 `r--`；`getfacl` 的 `#effective:` 比单独看到 named entry 更接近内核的最终判断。

概念继承、已有对象与新对象 构成 ACL 的时间维度。父目录的 default ACL 只初始化未来对象；已经存在的对象保留原 access ACL。协作目录因此必须分别处理当前目录、历史树和未来模板，并通过新文件、新目录以及不同身份的真实动作完成验证。

∴ {[.ops-quickref](#)}

操作语义以下入口分别观察 ACL 结构、修改当前规则、建立未来模板、复制或恢复元数据，并验证 `chmod` 与 mask 的交互。先理解命令作用对象，再记关键形式。

getfacl

SYNOPSIS

```
getfacl [-a|-d] [-e] [-p] file ...
getfacl -R -P -p directory
```

读取文件或目录的 access/default ACL、mask 与 effective 注释。它证明“对象存储了什么”，不能单独证明当前会话身份和最终业务动作。

重要参数 / 形式

getfacl -p PATH

保留绝对路径的前导 `/`，适合基线与审阅。

-a / -d

分别只显示 access ACL 或 default ACL，避免把当前规则与未来模板混读。

-e

强制显示所有受 mask 约束条目的 effective permissions。

-R -P

物理递归目录树，不跟随目录符号链接超出预期范围。

setfacl -m / -x / -b

SYNOPSIS

```
setfacl [-n|--mask] -m acl_spec file ...
setfacl -x acl_spec file ...
setfacl -b file ...
```

新增或修改 access ACL、删除指定条目，或清除扩展 access ACL。**-b** 是策略级清理，不应作为“不知道原因时”的默认排错动作。

重要参数 / 形式

-m u:alice:rw-

新增或修改 named user 条目。

-m g:qa:r--

新增或修改 named group 条目。

-m m::rw-

设置 group class 的统一上限；必须同时审阅其他受影响条目。

-x u:alice / -x g:qa

删除指定主体条目，不在删除规格中重复权限字段。

-b

清除 named user、named group、mask 等扩展 access ACL，保留基础 owner/group/other 条目。

-n / --mask

分别抑制 mask 重算，或强制根据条目重算 mask；两者都需要在完整 group class 证据下使用。

setfacl -d / -k

SYNOPSIS

```
setfacl -d -m acl_spec directory ...
setfacl -k directory ...
```

管理目录的 default ACL。default 只负责未来创建，不能证明当前目录或历史文件已经获得相同权限。

重要参数 / 形式

d:u:auditor:r-x

为未来对象模板加入 named user；目录通常需要穿越位。

d:g::rwx / d:m::rwx / d:o:---

明确 default owning group、default mask 与 default other。

-k

删除目录的 default ACL，不清除当前 access ACL。

新文件 / 新目录

必须分别创建并读取 ACL；普通文件不会因为模板中写了 **x** 就自动得到执行位。

ACL 复制、备份与恢复

SYNOPSIS

```
getfacl --access SOURCE | setfacl --set-file=- TARGET
getfacl -R -P -p DIRECTORY > acl.backup
setfacl --test --restore=acl.backup
setfacl --restore=acl.backup
```

把参考对象 ACL 复制到目标，或对目录树建立可审计的 ACL 元数据备份。恢复前应先预演，并确认备份中的路径、owner、group 与特殊位注释。

重要参数 / 形式

`--set-file=-`

从标准输入读取完整 ACL 并替换目标 ACL；不是增量追加。

`-R -P -p`

递归、物理遍历并保留绝对路径，适合目录树基线。

`--test --restore=FILE`

只显示恢复将产生的 ACL，不修改对象。

`--restore=FILE`

按备份恢复 ACL，并可能处理 owner、group 和特殊位；真实系统应先在测试树核对。

chmod 与 ACL 交互验证

SYNOPSIS

```
getfacl -e -p file
chmod g=MODE file
getfacl -e -p file
```

带扩展 ACL 的对象上，传统 group mode bits 通常映射 `mask::`。因此 `chmod g=...` 可能同时改变 named user、owning group 和 named group 的 effective 上限。

重要参数 / 形式

变更前后对比

对带 `+` 的对象执行 `chmod` 前后都运行 `getfacl -e`，不要只看 `ls -l`。

`ls -l` 的 group 位

在扩展 ACL 中通常显示 mask，而不一定直接等于 `group::` 条目。

最小修复

若目标只是恢复一个主体，先检查其他 group class 条目，避免通过放宽 mask 意外扩大授权。

[知识专题] ACL 不是“第四组权限”：先认清对象、条目与作用时点

传统模式位把访问者压缩为 owner、group、other 三类。ACL 没有推翻这套模型，而是在其上增加 named user、named group 和一个控制 group class 的 mask。理解 ACL 的最顺切入点不是背命令，而是把每一行映射到“谁、针对哪个对象、在何时生效”。

① [知识点] access ACL 管当前对象，default ACL 管未来创建

每个文件或目录都可以有 access ACL。目录还可以额外具有 default ACL：

access ACL
→ 现在访问这个文件或目录时参与判断

default ACL
→ 在目录内创建新对象时，用来初始化新对象的 access ACL

普通文件不能拥有 default ACL。目录的 default ACL 不参与该目录自身的访问检查，因此“目录上已经有 `default:user:auditor:r-x`”并不表示 auditor 现在就能进入这个目录；通常还需要相应的 access ACL。

② [知识点] 基础条目与扩展条目分别代表谁

一个完整 access ACL 可能包含：

文本条目	主体	是否需要 qualifier	是否受 mask 限制
<code>user::rwx</code>	文件所有者	否	否
<code>user:alice:rwx</code>	named user alice	是	是
<code>group::r-x</code>	文件所属组	否	是
<code>group:qa:rwx</code>	named group qa	是	是
<code>mask::r-x</code>	group class 上限	否	不适用
<code>other::---</code>	未匹配其他主体者	否	否

`user::` 中第二字段为空，表示文件所有者；`user:alice:` 的 qualifier 是 alice。`group::` 表示文件记录的 owning group，而不是“当前用户的主组”。

③ [知识点] 有 named user 或 named group 时，mask 成为必要结构

只包含 `user::`、`group::`、`other::` 的 ACL 可以直接映射传统模式位。出现 named user 或 named group 时，ACL 必须具有 mask。`setfacl` 通常会补齐必要条目并计算 mask，但这不意味着自动得到的上限一定符

合最终最小授权目标，仍要用 `getfacl` 检查。

④ [知识点] 目录 `rx` 仍保留传统目录语义

ACL 条目中的 `rx` 没有创造新的权限含义：

- 对普通文件，`r` 读取内容，`w` 修改内容，`x` 执行；
- 对目录，`r` 列出名称，`w` 修改目录项，`x` 穿越并访问已知名称。

因此给审计用户目录 `r--` 往往不够：他可能看到目录名，却无法访问其中已知文件。典型只读遍历授权是目录 `r-x`、普通文件 `r--`。

⑤ [知识点] ACL 属于自主访问控制的一层，不覆盖其他安全层

ACL 判断通过，只能说明这一层没有拒绝。路径上任一级目录缺少 `x`、只读挂载、应用自身限制或 SELinux 拒绝，都可能让功能测试失败。本章诊断会在 ACL 证据完成后指出分流方向，但不展开 SELinux 规则。

[Cheatsheet] 当前对象看 `access`；未来对象看 `default`；基础条目是 `owner/group/other`；`named user`、`owning group`、`named group` 都进入 `group class`；目录只读通常需要 `r-x`。

[知识专题] 从条目到实际权限：访问匹配、mask 与 effective

ACL 输出里最危险的阅读方式是只找到自己的名字，然后把该行右侧权限当作最终结果。内核先选择匹配的类别，再对 `group class` 条目应用 `mask`。某些匹配一旦发生，即使权限不足，也不会继续回退到看起来更宽的其他条目。

① [知识点] 文件所有者匹配后不会再使用组或 `other`

访问检查首先比较进程 `effective UID` 与文件 `owner UID`。若匹配，只检查 `user::`。如果 `owner` 条目不含所请求权限，访问被拒绝，不会把文件所有者再当作所属组成员或 `other` 重新计算。

这与传统权限“最具体类别优先”一致，也解释了为什么文件所有者权限较窄时，所属组较宽并不能自动补足。

② [知识点] `named user` 匹配后同时受自己的条目和 `mask` 约束

若进程不是文件所有者，但 `effective UID` 匹配 `user:<NAME>:`，则：

```
named-user effective
= named-user entry n mask
```

若 `user:analyst:rw-` 而 `mask::r--`，`analyst` 的有效权限是 `r--`。即便 `analyst` 也属于某个具有写权限的 `named group`，`named user` 已匹配，不能依靠组条目绕过 `named user` 的结果。

③ [知识点] 组阶段会考虑 `owning group` 和所有匹配 `named groups`

若没有 `owner` 或 `named user` 匹配，内核比较进程 `effective GID` 和所有 `supplementary groups`：

- 是否匹配文件 owning group;
- 是否匹配一个或多个 named group。

只要某个匹配的组条目与 mask 共同包含所请求权限，就可以通过该项访问。调查时不能只读第一条组记录，应结合用户实际组集合和所有匹配组条目。

④ [知识点] other:: 只用于没有匹配任何用户或组的情况

other:: 不是“最后再补一点权限”。只要前面的 owner、named user 或组阶段已匹配，就按该阶段决定允许或拒绝；只有完全没有匹配时才检查 other::。

⑤ [知识点] mask 是整个 group class 的统一上限

mask 约束以下条目：

```
user:<NAME>:
group::
group:<NAME>:
```

它不约束：

```
user::
other::
```

因此 `setfacl -m m::rwx` 不是“只修复 analyst”。它可能同时放宽所有 named user、owning group 和 named group 的 effective 上限。最小修复前必须查看整个 group class。

⑥ [知识点] 扩展 ACL 存在时，ls -l 的 group 位通常显示 mask

若 ACL 有 mask，传统 group mode bits 对应 mask::，而不是简单对应 group::。因此：

```
-rw-r-----+
```

中间三位 `r--` 可能表示 mask 是 `r--`，文件所属组条目本身也许写着 `rw-`，但 effective 被截断。`chmod g=...` 会修改 group mode bits，也就可能改变 mask，并同时影响多个 ACL 条目。

[Cheatsheet] owner → named user → 任一匹配组 → other; named user 和组类都受 mask; #effective: 才是被 mask 后的结果；扩展 ACL 下 `ls` 的 group 位通常是 mask。

[操作专题] 用 ls、stat 与 getfacl 建立可判定的 ACL 证据

查询阶段的目标不是收集更多输出，而是回答四个明确问题：目标对象是谁、当前 access ACL 是什么、目录是否有 default ACL、条目是否被 mask 截断。`ls` 适合快速筛选，最终判断应回到 `getfacl`。

① [操作] 用 `ls -ld` 发现 ACL 线索，但不据此推断授权细节

作用对象：文件或目录的模式摘要。基本形式：

```
ls -ld /srv/atlas /srv/atlas/plan.md
```

权限字符串末尾出现 `+`，说明对象具有扩展 access ACL 或目录具有 default ACL。它不能告诉你有哪些 named 条目、mask 是什么，也不能证明目标用户具有权限。

② [操作] 用 `getfacl -p` 查看完整对象视图

```
getfacl -p /srv/atlas
```

`-p` 保留绝对路径的前导 `/`，适合基线、备份和审阅。典型输出：

```
# file: /srv/atlas
# owner: root
# group: project
# flags: -s-
user::rwx
user:auditor:r-x
group::rwx
mask::rwx
other::---
default:user::rwx
default:user:auditor:r-x
default:group::rwx
default:mask::rwx
default:other::---
```

头部给出路径、owner、group；`# flags:` 可能显示 setuid、setgid 或 sticky 位。本章只使用它确认第 08 章已经建立的目录特殊位状态。

③ [操作] 用 `-a` 与 `-d` 分离当前状态和未来模板

```
getfacl -a -p /srv/atlas    # 只看 access ACL
getfacl -d -p /srv/atlas    # 只看 default ACL
```

把两者分开能防止将 `default:user:auditor:r-x` 错读成当前目录授权。普通文件执行 `getfacl -d` 不会得到可用的 default ACL，因为普通文件不能拥有它。

④ [操作] 用 `-e` 强制显示所有 effective 权限

默认情况下，只有条目权限与有效权限不同时才显示 `#effective:`。诊断时可强制显示：

```
getfacl -e -p /srv/atlas/plan.md
```

例如：

```
user:analyst:rw-          #effective:rw-
group::rw-                #effective:rw-
mask::r--
```

这条证据直接说明写权限被 mask 截断，而不是 named user 条目缺失。

⑤ [操作] 在名称解析和递归范围有疑问时切换查询方式

```
getfacl -n -p /srv/atlas/plan.md      # 显示数值 UID/GID
getfacl -R -P -p /srv/atlas           # 物理递归，不跟随目录符号链接
getfacl -R -P -s -p /srv/atlas        # 跳过只有基础 ACL 的对象
```

-R 会产生大量输出，应先确认目标根目录。-P 避免沿目录符号链接进入树外；-L 会逻辑跟随，除非确有需求，不应把它作为默认递归选择。

验证边界：getfacl 证明存储的 ACL 结构和 effective 计算，不能证明当前登录会话身份、父路径访问或最终业务动作。

帮助入口：man getfacl、getfacl --help。

[Cheatsheet] ls 的 + 只作提示；完整看 getfacl -p；当前与未来用 -a/-d 分开；mask 排错用 -e；递归优先 -R -P。

[操作专题] 修改当前对象的 access ACL：新增、修改、删除与 mask 控制

setfacl 的操作对象是文件或目录上已经存储的 ACL。最安全的工作方式是：先保存基线，只修改目标 entry，再立即读取 ACL 和进行功能测试。清空全部扩展条目虽然看起来简单，却可能删除仍然有效的历史授权。

① [操作] 用 -m 新增或修改 named user 和 named group

```
setfacl -m u:alice:rw- /srv/atlas/plan.md
setfacl -m g:qa:r-- /srv/atlas/plan.md
setfacl -m u:auditor:r-x /srv/atlas
```

-m 对不存在的条目执行新增，对已存在的同主体条目执行修改。目录是否需要 x 取决于目标动作；只读遍历通常使用 r-x。

一条命令可包含多个逗号分隔条目：

```
setfacl -m u:auditor:r--,g:qa:rw- /srv/atlas/plan.md
```

② [操作] 明确修改基础条目和 mask 的语义

```
setfacl -m g::rw- /srv/atlas/plan.md
setfacl -m m::rw- /srv/atlas/plan.md
setfacl -m o::--- /srv/atlas/plan.md
```

- `g::rw-` 修改 owning group 条目；
- `m::rw-` 修改整个 group class 的上限；
- `o::---` 修改未匹配其他主体者。

不要把 `g::` 与 `g:<GROUP>` 混淆。前者使用文件记录的 owning group，后者是额外 named group。

③ [操作] 理解自动 mask、`-n` 与 `--mask`

默认情况下，`setfacl` 会在需要时重新计算 mask，使其包含 owning group、所有 named user 和 named group 权限的并集；若同一操作显式给出了 mask，则默认尊重显式值。

```
setfacl -m u:analyst:rw- file           # 通常自动调整 mask
setfacl -n -m u:analyst:rw- file         # 不重新计算现有 mask
setfacl --mask -m u:analyst:rw-,m::r-- file # 即使显式给 mask 也重新计算
```

`-n` 不是通用“更安全”开关。保留过窄 mask 会让新条目看似成功但 effective 不足；自动重算又可能放宽其他条目的上限。正确做法是先读取整个 group class，再选择是否控制 mask。

④ [操作] 用 `-x` 删除一个主体条目

```
setfacl -x u:alice /srv/atlas/plan.md
setfacl -x g:qa /srv/atlas/plan.md
```

删除语法只描述 entry 类型和 qualifier，不应把权限字段作为目标。若目标条目不存在，命令可能报告无可删除的条目或保持不变；无论返回信息如何，都应再次用 `getfacl` 确认最终结构。

⑤ [操作] 区分 `-b` 与 `-k`

```
setfacl -b /srv/atlas/plan.md # 删除扩展 access ACL，保留 owner/group/other 基础条目
setfacl -k /srv/atlas         # 删除目录的 default ACL
```

`-b` 不等于“删除所有权限”，但会清除 named user、named group 和 mask 等扩展 access 条目；`-k` 只删除 default ACL。二者都是策略级变更，不应作为不知道原因时的排错捷径。

⑥ [操作] 用 `x` 表达目录穿越而避免给所有普通文件加执行位

`setfacl` 的权限字段支持大写 `X`：对象是目录，或该文件已经对某类主体具有执行位时，才设置执行权限。

```
setfacl -R -P -m g::rwX,u:auditor:r-X /srv/atlas
```

这适合树形目录的访问策略，但递归变更仍可能影响大量历史对象。高风险环境优先用 `find` 分开目录和普通文件，明确每类目标权限。

验证：

```
getfacl -e -p /srv/atlas/plan.md  
sudo -u alice test -w /srv/atlas/plan.md
```

这里的 `sudo -u` 只作为切换测试身份的工具；`sudo` 授权配置属于第 10 章。

帮助入口：`man setfacl`、`setfacl --help`。

[Cheatsheet] `-m` 新增/修改；`-x` 删除一条；`-b` 清扩展 access；`-k` 删 default；默认可能重算 mask，`-n` 保留现有 mask；递归先限定范围。

[操作专题] default ACL 与创建时继承：分别处理目录、新文件和新子目录

default ACL 的本质是创建模板。它解决“未来谁会得到什么初始 ACL”，而不是回溯修复历史。理解继承必须同时观察父目录模板、创建程序请求的 mode、新对象 access ACL，以及新子目录是否继续保存 default ACL。

① [知识点] 新对象首先继承模板，但不能超过创建程序请求的 mode

父目录存在 default ACL 时，新对象用它初始化 access ACL，然后移除创建程序所请求 mode 中没有的权限。常见程序创建普通文件时请求 `0666`，所以即使 default ACL 写有 `x`，普通新文件通常也不会凭空得到执行位；新目录常以 `0777` 为上限，因而可以继承穿越位。

父目录没有 default ACL 时，才使用传统 mode 与 umask 路径。default ACL 存在时，不应再用“简单地把 umask 从 0666/0777 中减掉”预测结果，最终必须创建样本验证。

② [操作] 为目录建立完整 default ACL

假设 `/srv/atlas` 的 owning group 已是 `project`，希望项目组协作、auditor 只读、其他人无权：

```
setfacl -m \  
d:u::rwX,d:u:auditor:r-X,d:g::rwX,d:m::rwX,d:o::--- \  
/srv/atlas
```

创建 named default user 后，default owner、group、other 与 default mask 必须构成有效 ACL。`setfacl` 可补齐部分必要条目，但显式写出目标模板更便于审阅。

③ [知识点] default ACL 不改变父目录自己的 access ACL

前一条命令只设置模板。如果 auditor 还不能进入 `/srv/atlas`，应单独配置当前目录 access ACL：

```
setfacl -m u:auditor:r-x,m::rwx /srv/atlas
```

建立协作目录时通常要成对考虑：

```
当前目录 access ACL
+
未来对象 default ACL
```

④ [验证点] 新文件与新目录必须分别创建并读取 ACL

```
sudo -u alice touch /srv/atlas/from-alice.txt
sudo -u alice mkdir /srv/atlas/from-alice.d

getfacl -p /srv/atlas/from-alice.txt
getfacl -p /srv/atlas/from-alice.d
```

预期判断：

- 新文件得到从父目录 default ACL 派生的 access ACL；
- 新文件通常不凭 default ACL 获得执行位；
- 新目录得到 access ACL，并继续具有可传递给更深层对象的 default ACL；
- 实际权限仍不得超过创建程序请求的 mode。

⑤ [边界] 已有对象不会因父目录新增 default ACL 而改变

```
getfacl -p /srv/atlas/old.txt
```

若 `old.txt` 在 default ACL 建立前已经存在，它不会自动获得 auditor 或 project 的新条目。必须根据题目范围单独设置其 access ACL，或对明确的已有树执行受控递归修改。

[Cheatsheet] default 是创建模板；父目录当前访问还要 access；新文件与新目录分别验证；普通文件不凭模板自动得到 `x`；旧对象必须另行处理。

[操作专题] 构造协作目录：把当前、历史、未来和真实身份连成闭

环

协作目录不是一条命令，而是一组相互独立的状态：目录所属组和 setgid 由第 08 章负责，ACL 负责额外主体和继承模板，功能测试负责证明真实用户能够完成预期动作。最常见的失败来自只配置其中一层。

① [操作] 先建立基线和授权矩阵

变更前记录：

```
ls -ld /srv/atlas
stat -c '%A %a %U %G %n' /srv/atlas
getfacl -R -P -p /srv/atlas > /root/atlas-acl.before
```

再把要求写成矩阵：

身份	目录	普通文件	目标动作
project 成员	<code>rwX</code>	<code>rw-</code>	创建、读取、修改
auditor	<code>r-X</code>	<code>r--</code>	遍历、读取
other	<code>---</code>	<code>---</code>	拒绝

矩阵先于命令，可以防止把“只读”误写成目录 `r--`，也能明确拒绝测试。

② [操作] 分别修复当前目录和已有树

当前目录：

```
setfacl -m g::rwX,u:auditor:r-X,m::rwX,o::--- /srv/atlas
```

对已有树，先预览范围，再按类型设置：

```
find /srv/atlas -xdev -type d -print
find /srv/atlas -xdev -type f -print

find /srv/atlas -xdev -type d -print0 \
  | xargs -0 -r setfacl -m g::rwX,u:auditor:r-X,m::rwX,o::---

find /srv/atlas -xdev -type f -print0 \
  | xargs -0 -r setfacl -m g::rw-,u:auditor:r--,m::rw-,o::---
```

这里按目录和普通文件分开，避免误给普通文件执行位。真实环境还应评估是否包含需要保持可执行的脚本；若有，应按题意保留而不是机械覆盖。

③ [操作] 为未来对象建立 default ACL

```
setfacl -m \
  d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:--- \
  /srv/atlas
```

如果已有子目录也必须继续继承相同模板，需要在明确范围后为这些目录设置 default ACL；父目录新加模板不会回溯给旧子目录。

④ [验证点] 使用两个协作身份交叉写入

```
sudo -u alice sh -c 'printf "%s\n" alpha > /srv/atlas/team.txt'
sudo -u bob sh -c 'printf "%s\n" beta >> /srv/atlas/team.txt'
getfacl -e -p /srv/atlas/team.txt
```

alice 创建成功只能证明 alice 具有创建能力；bob 能追加才证明新文件的 group class 具有实际写权限，并且 bob 当前会话具有正确组身份。

⑤ [验证点] 同时验证 auditor 的允许与拒绝动作

```
sudo -u auditor cat /srv/atlas/team.txt
sudo -u auditor test ! -w /srv/atlas/team.txt
sudo -u auditor sh -c 'echo denied >> /srv/atlas/team.txt' # 应失败
sudo -u auditor touch /srv/atlas/denied.txt # 应失败
```

拒绝测试是最小特权验收的一部分。只验证“能读”而不验证“不能写”，可能遗漏过宽 mask 或错误的目录写权限。

[Cheatsheet] 基线 → 授权矩阵 → 当前目录 → 已有树 → default 模板 → 两个协作身份交叉写 → 只读身份允许/拒绝测试。

[操作专题] 复制、备份与恢复 ACL：让批量变更可审计、可预演

ACL 是文件元数据。复制一个配置文件的内容不会自动表达“把这个对象的 ACL 策略复制到另一个对象”。批量变更前应保留 ACL 基线；恢复时还要理解备份中可能包含 owner、group 和特殊位注释，不能把恢复当成无影响的单条 ACL 修改。

① [操作] 从参考对象复制 access ACL

```
getfacl --access /srv/atlas/reference.txt \
| setfacl --set-file=- /srv/atlas/target.txt
```

`getfacl` 输出可作为 `setfacl` 输入，注释行会被忽略。`--set-file` 是替换目标 ACL，不是增量添加；执行前应比较目标现有授权，避免静默覆盖仍需保留的 `named` 条目。

② [操作] 在确实相同的策略下，把 access ACL 转为目录 default ACL

```
getfacl --access /srv/atlas \  
| setfacl -d -M- /srv/atlas
```

`-d` 将输入的普通条目应用为 default 条目。只有当“当前目录 access 策略”确实适合作为“未来对象模板”时才使用；目录当前可写不代表普通新文件也应该可执行。

③ [操作] 递归备份完整目录树 ACL

```
getfacl -R -P -p /srv/atlas > /root/atlas-acl.backup
```

- `-R` 递归；
- `-P` 物理遍历，避免沿目录符号链接越界；
- `-p` 保留绝对路径。

备份文件应作为变更证据保存，并检查非空、路径正确、包含预期对象：

```
test -s /root/atlas-acl.backup  
grep -F '# file: /srv/atlas' /root/atlas-acl.backup
```

④ [操作] 用 `--test` 预演恢复结果

```
setfacl --test --restore=/root/atlas-acl.backup
```

测试模式只列出将得到的 ACL，不修改对象。预演输出仍需人工核对范围、路径、owner/group 注释和特殊位影响。

⑤ [操作] 执行恢复并分层验证

```
setfacl --restore=/root/atlas-acl.backup  
getfacl -R -P -p /srv/atlas > /root/atlas-acl.after-restore
```

`--restore` 可尝试恢复 owner、owning group，并依据 flags 注释设置 `setuid`、`setgid`、`sticky` 位；若输入没有 flags 注释，相关特殊位可能被清除。因此在生产系统上应优先在副本或测试树验证恢复文件，而不是把它当作仅修改 `named` ACL 的轻量命令。

⑥ [边界] 内容归档与 ACL 元数据不是同一个问题

`cp`、`mv`、`tar`、`rsync` 是否保留 ACL 取决于具体命令、选项和目标文件系统。这些工具的完整语义归第 06 章《复制、归档、压缩与远程传输》。本章只要求：迁移或恢复后必须重新 `getfacl`，不能根据“文件已经复制成功”推断 ACL 也已保留。

[Cheatsheet] 复制 ACL 用 `getfacl` 管道；替换前先比较；备份用 `-R -P -p`；恢复先 `--test`；`--restore` 可能连 `owner/group/特殊位` 一起处理；内容成功不等于 ACL 保留。

[诊断专题] ACL 条目明明存在，为什么用户仍然 `Permission denied`

诊断不能从“再加一个 `rw`”开始。应从症状进入最有区分度的证据，确定失败位于身份、路径、ACL 匹配、`mask`、继承时点还是其他安全层。每一步都只做能够解释当前证据的最小修复。

① [诊断点] 先确认实际进程身份，而不是只看账号数据库

```
id analyst
sudo -u analyst id
```

账号刚加入新组时，已存在的 Shell 可能仍使用旧的 `supplementary groups`。若测试进程没有预期组，继续修改 ACL 会掩盖身份问题。应创建新会话或重新登录后再验证。

② [诊断点] 条目存在但 `#effective:` 缺少所需权限

症状：

```
user:analyst:rw-          #effective: r--
mask::r--
```

假设：近期 `chmod g=r` 或显式 `mask` 修改收窄了 `group class`。下一条证据：

```
getfacl -e -p /srv/atlas/report.txt
```

最小修复前先检查所有受 `mask` 影响的条目。若只需要恢复 `analyst` 写入，而其他条目本来就无 `w`，可以提高 `mask` 上限；若其他条目声明了 `w` 但业务不应生效，应先收窄那些条目，再调整 `mask`。

③ [诊断点] `default ACL` 正确，但失败对象是历史文件

症状：新文件可读，旧文件不可读。证据：

```
getfacl -d -p /srv/atlas
getfacl -p /srv/atlas/old.txt
getfacl -p /srv/atlas/new.txt
```

若模板只出现在目录和新文件，说明 default ACL 工作正常，历史对象没有回溯。最小修复是给明确的旧对象或已有树设置 access ACL，而不是反复修改 default ACL。

④ [诊断点] 目录 ACL 有权限，但父路径不可穿越

```
namei -l /srv/atlas/report.txt
```

任何一级父目录缺少目标身份的 `x`，最终文件 ACL 再宽也不可达。`namei` 与传统目录权限完整语义属于第 08 章，本章只把它作为 ACL 诊断前置证据。

⑤ [诊断点] `chmod` 修复了一个现象，却改变了整个 group class

扩展 ACL 存在时，执行：

```
chmod g=r /srv/atlas/report.txt
```

可能把 mask 改为 `r--`，导致多个 named user/group 同时失去写权限。变更后应立刻比较：

```
ls -l /srv/atlas/report.txt  
getfacl -e -p /srv/atlas/report.txt
```

不要只看到 `ls` group 位符合预期就结束，因为它可能表示 mask，而不是 owning group 条目本身。

⑥ [诊断点] ACL 和路径均允许时，转入下一安全层

若：

- 测试进程身份正确；
- 每级父目录可穿越；
- access ACL 匹配且 effective 包含所需权限；
- 文件系统不是只读；

仍失败，应保留这些证据并转向 SELinux、应用策略、挂载或其他层。不要用 `chmod 777` 或清空 ACL 来“验证”SELinux，因为这会破坏原有授权且通常不能解决非 DAC 拒绝。

诊断链：

症状

- 当前 id
- 父路径 namei
- getfacl 选择条目与 mask
- 区分旧对象/新对象
- 检查 chmod 历史
- 最小修复
- 真实身份再验证
- 其他安全层分流

[Cheatsheet] 先身份，再路径；条目存在看 effective；模板正确看对象创建时点；chmod 可能收窄 mask；ACL 证据闭环后再转 SELinux。

[经典任务] 维护一个已有内容的项目协作目录

环境

系统中已存在：

```
组: project
成员: alice、bob
只读审计用户: auditor
目录: /srv/atlas
目录所有者: root
目录所属组: project
目录模式: 2770
已有对象:
    /srv/atlas/plan.md
    /srv/atlas/archive/q1.txt
```

第 08 章前置状态已经成立：账号与组成员关系正确、`/srv/atlas` 具有 setgid、父路径可穿越。本任务不求创建用户、修改 sudoers 或处理 SELinux。

当前状态

- `/srv/atlas` 没有 default ACL；
- alice 能创建文件，但 bob 不一定能修改 alice 创建的新文件；
- auditor 无法读取历史文件；
- 已有文件不得删除、重建或覆盖；
- 变更前尚未保存 ACL 基线。

目标终态

1. project 组成员能进入 `/srv/atlas`，创建、读取和修改项目文件；
2. alice 创建的新文件能被 bob 追加；

- 3. 新建子目录继续继承同一协作策略；
- 4. auditor 能遍历目录并读取普通文件，但不能修改、创建或删除；
- 5. 已有的 plan.md 与 archive/q1.txt 也满足 auditor 只读；
- 6. other 不获得访问权；
- 7. 保存可用于恢复的 ACL 备份。

限制条件

- 不得使用 chmod 777 ；
- 不得清空全部 ACL 后重新配置；
- 不得删除或重建已有对象；
- 递归修改前必须列出目标范围；
- 不得把命令成功或 getfacl 输出代替多身份功能测试；
- 不能声称已在本会话的 RHEL 9 VM 中实测。

验收证据

层次	必须提交的证据
基线	ls -ld、stat、递归 ACL 备份
当前目录	access ACL 与 default ACL
已有对象	两个指定文件的 getfacl -e
新文件	alice 创建、bob 追加、ACL 输出
新目录	access ACL 与继续存在的 default ACL
auditor 允许	读取已有文件和新文件成功
auditor 拒绝	修改文件、创建文件失败
范围	递归操作未越出 /srv/atlas

请先独立完成。参考解答从下一页开始。

[参考解答] 维护已有项目协作目录

以下命令是依据题目环境给出的参考操作路径。本章未执行 RHEL 9 命令级实机验证，所有输出均应在真实练习环境中取得，不应照抄虚构结果。

① [调查] 确认基线、身份和对象范围

```
id alice
id bob
id auditor

ls -ld /srv/atlas
stat -c '%A %a %U %G %n' /srv/atlas
find /srv/atlas -xdev -printf '%y %p\n'

getfacl -R -P -p /srv/atlas > /root/atlas-acl.before
test -s /root/atlas-acl.before
```

判断：

- `id` 证明账号数据库解析出的组身份；真实测试仍要用新进程；
- `2770 root:project` 属于第 08 章前置，不在本任务重建；
- `-xdev` 限制 `find` 不跨文件系统，`-P` 限制 ACL 递归不跟随目录符号链接；
- 备份在变更前生成，便于审计和恢复。

② [操作] 配置当前目录 access ACL

```
setfacl -m g::rwx,u:auditor:r-x,m::rwx,o::--- /srv/atlas
```

参数解释：

- `g::rwx`：owning group project 对当前目录完全协作；
- `u:auditor:r-x`：auditor 可列出和穿越，但无目录写权限；
- `m::rwx`：group class 上限允许项目组写入；auditor 自身条目没有 `w`，所以不会因此获得写权限；
- `o::---`：其他主体无权。

③ [操作] 修复已有目录和普通文件

先确认类型：

```
find /srv/atlas -xdev -type d -print
find /srv/atlas -xdev -type f -print
```

再分类型修改：

```
find /srv/atlas -xdev -type d -print0 \
| xargs -0 -r setfacl -m g::rwx,u:auditor:r-x,m::rwx,o::---

find /srv/atlas -xdev -type f -print0 \
| xargs -0 -r setfacl -m g::rw-,u:auditor:r--,m::rw-,o::---
```

这里没有删除或重建任何文件。目录与普通文件分开，避免给所有普通文件增加执行位。若树中存在原本必须可执行的脚本，应在真实任务中按清单单独保留其执行权限，而不是机械套用第二条命令。

④ [操作] 给当前和已有子目录设置 default ACL

题目要求新子目录继续继承策略。根目录先设置：

```
setfacl -m \  
  d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:--- \  
  /srv/atlas
```

如果已有 `/srv/atlas/archive` 也会直接接收新对象，应给已有目录设置同样模板：

```
find /srv/atlas -xdev -type d -print0 \  
  | xargs -0 -r setfacl -m \  
    d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:---
```

这一步只对目录设置 default ACL；普通文件不能拥有 default ACL。

⑤ [结构验证] 读取当前、历史和模板状态

```
getfacl -a -e -p /srv/atlas  
getfacl -d -e -p /srv/atlas  
getfacl -e -p /srv/atlas/plan.md  
getfacl -e -p /srv/atlas/archive/q1.txt  
getfacl -d -e -p /srv/atlas/archive
```

判断重点：

- project 对目录 effective 包含 `rwX`，对普通文件包含 `rw-`；
- auditor 对目录 effective 为 `r-x`，对普通文件为 `r--`；
- other 为 `---`；
- root 和已有子目录具有预期 default ACL；
- 没有 `#effective:` 暴露意外的 mask 截断。

⑥ [功能验证] 创建新文件、新目录并交叉写入

```
sudo -u alice sh -c 'printf "%s\n" alpha > /srv/atlas/from-alice.txt'
sudo -u bob sh -c 'printf "%s\n" beta >> /srv/atlas/from-alice.txt'

sudo -u alice mkdir /srv/atlas/from-alice.d
sudo -u alice touch /srv/atlas/from-alice.d/nested.txt

getfacl -e -p /srv/atlas/from-alice.txt
getfacl -e -p /srv/atlas/from-alice.d
getfacl -d -e -p /srv/atlas/from-alice.d
getfacl -e -p /srv/atlas/from-alice.d/nested.txt
```

分层含义：

- alice 创建证明目录允许项目成员创建；
- bob 追加证明新文件 group class 的 effective 写权限正确；
- 新子目录同时检查 access 和 default ACL，证明策略可以继续向下传递；
- `getfacl` 仍不能替代实际写入，因此两类证据都要保留。

⑦ [功能验证] auditor 的允许与拒绝

```
sudo -u auditor cat /srv/atlas/plan.md
sudo -u auditor cat /srv/atlas/from-alice.txt
sudo -u auditor cat /srv/atlas/from-alice.d/nested.txt

sudo -u auditor sh -c 'echo denied >> /srv/atlas/from-alice.txt' # 预期失败
sudo -u auditor touch /srv/atlas/denied.txt # 预期失败
```

不能只执行 `test ! -w` 就结束，因为实际应用动作能同时暴露父路径、ACL 和创建/打开语义。失败命令应在练习环境中确认返回非零，并记录错误对象。

⑧ [恢复准备] 生成变更后备份并预演恢复

```
getfacl -R -P -p /srv/atlas > /root/atlas-acl.after
setfacl --test --restore=/root/atlas-acl.before
```

`--test` 只预演。真正执行 `--restore` 前应确认它可能恢复 owner、group 和特殊位。若本任务只要求保留回滚依据，不应为了“证明恢复可用”而破坏已经完成的目标状态。

典型错误

1. 只设置 default ACL，导致当前目录和历史文件仍不可访问；
2. 只对根目录设置 access ACL，忽略历史子目录和普通文件；
3. 将 auditor 目录权限写成 `r--`，导致无法穿越；

4. 把 mask 无条件设为 `rwX`，却没有检查其他 named 条目是否因此被放宽；
5. 用一条递归 `rwX` 给所有普通文件添加执行位；
6. 只检查 `getfacl`，没有用 `alice`、`bob`、`auditor` 做允许和拒绝测试；
7. 没有在变更前备份 ACL。

[经典任务] 诊断 named user 有 `rw-` 但实际无法写入

环境与症状

`analyst` 需要修改 `/srv/atlas/report.txt`。当前：

```
-rw-r-----+ 1 root project ... /srv/atlas/report.txt

user::rw-
user:analyst:rw-          #effective:r--
group::rw-                #effective:r--
group:qa:r--
mask::r--
other::---
```

`analyst` 能读取但不能追加。管理员回忆近期执行过：

```
chmod g=r /srv/atlas/report.txt
```

目标终态

- `analyst` 恢复读写；
- `project` owning group 仍可读写；
- `qa` 仍然只读；
- `other` 仍无权；
- 说明 `ls -l` group 位变化的原因；
- 以 `analyst` 实际身份完成追加验证。

限制条件

- 不得删除全部 ACL；
- 不得无调查执行 `mask::rwX`；
- 不得改变 owner 或 owning group；
- 不得给 `qa` 写权限；
- 必须保留修复前后 `getfacl -e` 证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 修复被 `chmod` 收窄的 mask

① [当前证据] 确认身份和实际失败

```
sudo -u analyst id
sudo -u analyst sh -c 'echo probe >> /srv/atlas/report.txt'
```

若测试进程身份不正确，应先修复会话身份；本题已知 `analyst` 能读且 `named user` 匹配，继续调查 ACL effective。

② [区分性证据] 强制显示所有 effective 权限

```
getfacl -e -p /srv/atlas/report.txt
```

关键推理：

```
analyst 条目声明 rw-
n mask r--
= effective r--
```

owning group 的 `rw-` 也被同一 mask 截断为 `r--`。qa 条目本身只有 `r--`，即使 mask 提高到 `rw-`，qa 仍不会获得 `w`。

③ [最小修复] 把 group class 上限恢复到 `rw-`

```
setfacl -m m::rw- /srv/atlas/report.txt
```

为什么不是 `rwX`：普通数据文件不需要执行，`rw-` 已足以满足 `analyst` 和 `project` 写入，也不会扩大执行权限。

④ [再验证] 比较结构、模式摘要和功能

```
getfacl -e -p /srv/atlas/report.txt
ls -l /srv/atlas/report.txt

sudo -u analyst sh -c 'echo fixed >> /srv/atlas/report.txt'
sudo -u analyst tail -n 1 /srv/atlas/report.txt
```

预期逻辑：

- `user:analyst:rw-` 不再显示被截断为只读；
- `group::rw-` effective 恢复为 `rw-`；
- `group:qa:r--` 仍只有只读，因为其条目本身没有 `w`；
- `ls -l` 的 group 位从 `r--` 变为 `rw-`，表示扩展 ACL 中的 mask 发生变化，不等同于只修改了 `group::`；
- `analyst` 实际追加成功。

⑤ [根因说明] `chmod g=r` 为什么影响 named user

扩展 ACL 有 mask 时，传统 group mode bits 对应 mask。`chmod g=r` 把 group 位设置为只读，也就把 `mask::` 收窄为 `r--`。named user `analyst`、owning group `project` 和 named group `qa` 都属于 group class，因此它们的 effective 上限一起变化。

这不是 `chmod` 与 ACL 相互独立，而是两者存在双向映射。以后对带 `+` 的对象执行 `chmod` 后，应立即运行 `getfacl -e`，检查是否改变了 named 条目的有效权限。

[本章收束] 把 ACL 还原成声明、有效、创建与功能四层证据

ACL 的稳定工作方法不是“先加 `rwX`”，而是从对象与证据推进：

建立基线与对象范围

- 分离 access ACL 和 default ACL
- 识别主体匹配与 mask
- 对当前、历史、未来状态做最小修改
- 用结构证据验证存储结果
- 用真实身份验证允许动作与拒绝动作
- 保存恢复依据

章末检查清单

- ☐ 已明确目标是当前对象、已有树还是未来新对象；
- ☐ 已分别读取 access ACL 和 default ACL；
- ☐ 已确认目标身份会匹配 owner、named user、group class 或 other；
- ☐ 已检查 `mask::` 和 `#effective::`；
- ☐ 已评估 `chmod` 是否改变 mask；
- ☐ 递归操作前已列出范围，并避免跟随目录符号链接越界；
- ☐ 新文件和新目录已分别验证继承；
- ☐ 已用至少两个协作身份完成交叉写入；
- ☐ 已验证只读主体既“能读”又“不能写/创建”；
- ☐ 已保存 ACL 基线或恢复文件，并理解 `--restore` 的影响范围。

主要判断表

观察或症状	首先说明什么	下一条最有区分度的证据	不能直接证明什么
<code>ls -l</code> 末尾出现 <code>+</code>	存在扩展 access ACL 或 default ACL	<code>getfacl -p</code>	具体主体与实际权限
<code>user:analyst:rw-</code>	名义条目包含读写	<code>getfacl -e -p</code>	effective 一定包含写
<code>default:user:auditor:r-x</code>	未来对象模板含 auditor	新建对象后 <code>getfacl</code>	父目录和旧文件已授权
新文件可协作、旧文件失败	default 可能正常	对比旧/新对象 access ACL	应继续修改 default ACL
<code>chmod g=r</code> 后 named user 失写	mask 可能被收窄	变更前后 <code>getfacl -e</code>	只有 owning group 被修改
ACL 结构正确仍拒绝	失败可能不在 ACL 层	<code>id</code> 、 <code>namei -l</code> 、挂载状态，再转其他安全层	不能据此认定放宽为 <code>chmod 777</code> 是正确修复
<code>setfacl</code> 返回成功	修改请求被接受	重新 <code>getfacl</code> + 真实身份动作	业务终态正确

向下一章交接

本章解决的是“对象本身允许谁访问”的自主访问控制。下一章《sudo 与最小特权授权》处理的是“某个已登录用户可以代表谁执行哪些管理命令”。ACL 不授予提权能力，sudo 也不替代目标文件和目录的 ACL；两者分别控制数据访问边界与管理命令边界。

静态可信声明：本章依据 RHEL 9 课程资料以及 `acl(5)`、`getfacl(1)`、`setfacl(1)` 进行静态核对；未执行命令级实机验证，文中命令均作为推荐操作与验证路径。